

Case Study: StreamFlix – Enterprise Video Streaming Platform Migration to AWS

1. Introduction

Company Overview

StreamFlix is a large-scale global video streaming platform delivering:

- Movies
- TV shows
- Live events
- Personalized recommendations
- Subscription-based streaming services

The platform initially operated using traditional on-premise infrastructure hosted in private datacenters.

As the company expanded globally, the legacy infrastructure became increasingly difficult to maintain and scale.

2. Problems in Existing On-Premise Infrastructure

2.1 Scalability Limitations

Traffic patterns became highly unpredictable.

Examples:

- New movie releases
- Weekend traffic spikes
- Live sports streaming
- Regional traffic surges

The existing infrastructure required manual server provisioning, leading to slow scaling.

2.2 Frequent Downtime

Infrastructure failures caused:

- Service interruptions
- High latency
- Failed playback sessions
- Login failures

Single points of failure existed across networking and compute layers.

2.3 Operational Complexity

Infrastructure management included:

- Hardware maintenance
- Firewall management
- Capacity planning
- Manual deployments
- Load balancing configuration

Engineering productivity was significantly reduced.

2.4 High Capital Expenditure

The organization continuously invested in:

- Physical servers
- Cooling systems
- Networking hardware
- Datacenter expansion

Costs increased regardless of actual usage.

3. Migration Goals

The company migrated to AWS to achieve the following objectives.

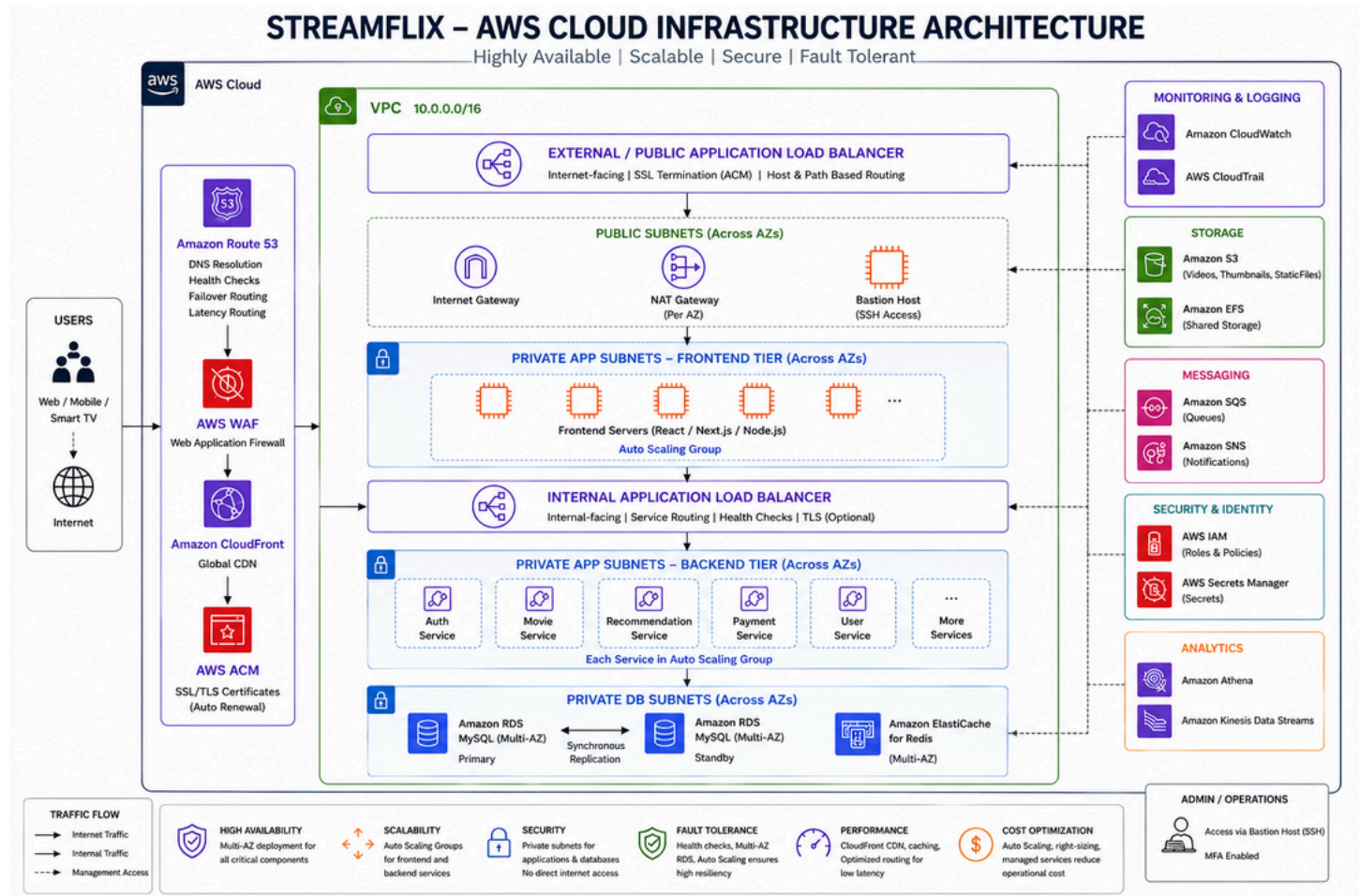
Goal	Description
High Availability	Eliminate downtime
Scalability	Automatically scale infrastructure
Security	Isolate internal services
Fault Tolerance	Survive infrastructure failures
Performance	Reduce latency globally
Cost Optimization	Pay only for utilized resources

4. Target AWS Architecture

StreamFlix adopted a modern cloud-native microservices architecture using:

- Multi-AZ deployment
- Public and private subnets
- External and internal load balancers
- Auto Scaling
- Managed databases
- CDN integration
- Internal service routing

The infrastructure follows a secure 3-tier enterprise architecture.



5. High-Level Architecture Overview

Users



CloudFront CDN



AWS WAF



Public ALB



Frontend Tier (Private Subnet)



Internal ALB



Backend Microservices



Database Layer

6. Amazon VPC Design

VPC Configuration

Component	Value
VPC CIDR	10.0.0.0/16
Architecture	Multi-tier
Deployment	Multi-AZ
Internet Connectivity	IGW + NAT Gateway

The VPC provides isolated networking for all AWS resources.

7. Subnet Architecture

The VPC is divided into multiple subnet layers.

7.1 Public Subnets

Public subnets contain internet-facing infrastructure.

Components

- Public Application Load Balancer
- NAT Gateway
- Bastion Host

These subnets have routes to the Internet Gateway.

7.2 Private Application Subnets

These contain:

- Frontend servers
- Backend microservices
- Internal APIs

These instances do not have public IP addresses.

7.3 Private Database Subnets

Database infrastructure is completely isolated.

Contains:

- Amazon RDS
- Redis cache
- Internal replication services

No internet access is allowed.

8. Internet Gateway (IGW)

The Internet Gateway allows communication between the VPC and the public internet.

Used by:

- Public ALB
- NAT Gateway
- Bastion Host

Public route table:

Destination	Target
0.0.0.0/0	Internet Gateway

9. NAT Gateway

Private instances require outbound internet access for:

- Security updates
- Downloading packages
- External API communication

The NAT Gateway enables outbound-only internet communication.

Private EC2

↓

NAT Gateway

↓

Internet

Inbound internet traffic remains blocked.

10. BASTION HOST IMPLEMENTATION

Why Bastion Host is Needed in StreamFlix

In the StreamFlix architecture, all frontend and backend EC2 instances are placed inside private subnets. These servers do not have public IP addresses because exposing application servers directly to the internet is dangerous in production.

However, DevOps engineers still need administrative access for:

- troubleshooting
- deployments
- checking logs
- restarting services
- debugging failures
- installing patches

Since private EC2 instances cannot be accessed directly from the internet, StreamFlix uses a Bastion Host.

The Bastion Host acts as a controlled entry point into the private infrastructure.

Actual Access Flow

DevOps Engineer Laptop

↓

Public Internet

↓

Bastion Host (Public Subnet)

↓

Frontend/Backend EC2 (Private Subnets)

Why This Architecture Is Important

Without Bastion Host:

- every private EC2 would need public IP
- SSH port 22 would be exposed publicly
- attack surface becomes huge
- difficult to audit access
- harder to control permissions

With Bastion Host:

- only one server accepts SSH
- private servers stay hidden
- centralized monitoring possible
- security groups become simpler
- easier compliance and auditing

STEP-BY-STEP IMPLEMENTATION

STEP 1 — Create Bastion Security Group

Go to:

AWS Console

↓

EC2

↓

Security Groups

↓

Create Security Group

Configuration:

Name:

streamflix-bastion-sg

Inbound Rules:

Type Port Source

SSH 22 Your Public IP

IMPORTANT:

Do NOT allow:

0.0.0.0/0

for SSH in production.

Why:

That exposes SSH to the whole internet.

Correct production practice:

Allow only office IP/VPN IP.

Outbound:

Allow all outbound traffic.

Reason:

Bastion needs to communicate with private EC2.

STEP 2 – Launch Bastion EC2 Instance

Go to:

EC2



Launch Instance

Configuration:

Name:

streamflix-bastion

AMI:

Ubuntu 22.04 or Amazon Linux

Instance Type:

t2.micro or t3.micro

Subnet:

Public-Subnet-A

Auto Assign Public IP:

Enabled

Security Group:

streamflix-bastion-sg

Key Pair:

Select existing key pair

Why public subnet:

Because Bastion must be reachable from internet.

Why public IP:

Without public IP, DevOps engineers cannot SSH into it.

STEP 3 – Configure Private EC2 Security Groups

Now configure frontend/backend security groups.

Frontend Security Group:

Allow SSH ONLY from Bastion SG.

Backend Security Group:

Allow SSH ONLY from Bastion SG.

Example:

Inbound Rule:

Type Port Source

SSH 22 streamflix-bastion-sg

Why security-group-to-security-group communication matters:

It is more secure than using IP addresses because:

- dynamic
- scalable
- AWS-managed trust relationship

STEP 4 – Verify Bastion Connectivity

From local laptop:

```
ssh -i streamflix.pem ubuntu@<BASTION_PUBLIC_IP>
```

If successful:

You are now inside Bastion.

STEP 5 – SSH from Bastion to Private EC2

Inside Bastion:

```
ssh -i streamflix.pem ubuntu@10.0.11.25
```

Now you can access private EC2 instances.

Why this works:

Private EC2 instances are reachable internally inside VPC.

11. CLOUDFRONT CDN IMPLEMENTATION

Why CloudFront Is Needed in StreamFlix

StreamFlix serves users globally across:

- web browsers
- mobile apps
- smart TVs

If every request directly hits the frontend servers in one AWS region:

- latency increases
- frontend servers become overloaded
- streaming performance drops
- static content loads slowly

CloudFront solves this by caching content at AWS edge locations closer to users.

Traffic Flow

User

↓

CloudFront Edge Location

↓

Public ALB

↓

Frontend Servers

What CloudFront Is Used For in StreamFlix

CloudFront caches and delivers:

- static frontend assets
- JavaScript files
- CSS files
- images
- movie thumbnails
- cached API responses
- streaming manifests

Benefits

- Reduced origin load
- Faster global access
- Lower latency
- Better streaming experience
- Improved scalability

STEP-BY-STEP IMPLEMENTATION

STEP 1 — Open CloudFront

Go to:

AWS Console

↓

CloudFront

↓

Create Distribution

STEP 2 — Configure Origin

Origin Domain:

Select Public ALB DNS name.

Example:

streamflix-public-alb.ap-south-1.elb.amazonaws.com

Why:

CloudFront forwards requests to ALB when content is not cached.

STEP 3 — Configure Viewer Protocol Policy

Select:

Redirect HTTP to HTTPS

Why:

Forces secure communication.

STEP 4 — Configure Cache Behaviors

Example configuration:

Path Pattern Behavior

/static/* Long cache TTL

/images/* Aggressive caching

/api/* Minimal caching

/videos/* Optimized delivery

Why:

Different content requires different cache policies.

STEP 5 — Enable Compression

Enable:

- **Gzip**
- **Brotli**

Why:

Reduces file size and improves loading speed.

STEP 6 – Attach ACM SSL Certificate

Under:

Custom SSL Certificate

Select:

ACM certificate for streamflix.com

Why:

Enables HTTPS at CloudFront edge.

IMPORTANT:

**CloudFront ACM certificates must exist in:
us-east-1 region**

STEP 7 – Configure Alternate Domain Names

Add:

www.streamflix.com

Optional:

media.streamflix.com

Why:

Users access branded domains instead of CloudFront-generated URLs.

STEP 8 – Configure Route53

Create Alias Record:

www.streamflix.com

↓

CloudFront Distribution

Traffic now becomes:

User

↓

Route53

↓

CloudFront

↓

Public ALB

STEP 9 – Validate CloudFront

Open:

<https://www.streamflix.com>

Verify:

- HTTPS works
- static files load quickly
- CloudFront cache headers appear

Example header:

x-cache: Hit from cloudfront

means content is being cached successfully.

Production Optimizations

Large streaming platforms also use:

- signed URLs
- geo restriction
- cache invalidation
- Origin Shield
- Lambda@Edge
- WAF integration

How CloudFront Fits Into StreamFlix Architecture

Users

↓

Route53

↓

CloudFront

↓

WAF

↓

Public ALB

↓

Frontend Services

Why CloudFront Is Important for StreamFlix

Without CloudFront:

- higher latency
- overloaded frontend servers

- slower content delivery
- poor global performance

CloudFront improves:

- scalability
- speed
- caching efficiency
- global user experience

For large streaming platforms, CDN is a core infrastructure component.

12. AWS WAF

Why AWS WAF Is Needed in StreamFlix

StreamFlix is a public internet-facing platform. Millions of users access the application daily. The moment a service becomes public, it automatically becomes a target for:

- bots
- scraping
- SQL injection attempts
- API abuse
- brute-force attacks
- fake traffic floods
- DDoS attacks
- credential stuffing
- malicious crawlers

Without protection, attackers can:

- overload frontend servers
- exploit APIs
- bypass authentication
- consume infrastructure resources
- increase AWS cost
- cause downtime

AWS WAF acts as the first intelligent filtering layer before traffic reaches the application infrastructure.

Traffic Flow with WAF

User Request

↓

CloudFront

↓

AWS WAF

↓

Public ALB

↓

Frontend Servers

What WAF Actually Does

WAF inspects incoming HTTP/HTTPS requests.

It checks:

- request headers
- query strings
- request body
- source IP
- country
- request rate
- known attack signatures

Then it decides:

- Allow
- Block
- Count
- CAPTCHA challenge

Why StreamFlix Attaches WAF to CloudFront

Best production practice:

Attach WAF at CloudFront layer.

Reason:

Traffic gets filtered BEFORE entering AWS regional infrastructure.

Benefits:

- reduced backend load
- lower ALB traffic
- earlier attack mitigation
- lower infrastructure cost

Alternative:

Attach WAF directly to ALB.

Still valid.

But filtering happens later in request lifecycle.

Best architecture for StreamFlix:

User



CloudFront



WAF



Public ALB

Why WAF Is Critical Specifically for StreamFlix

Streaming platforms are high-value targets because:

- **huge public traffic**
- **login systems**
- **payment APIs**
- **subscription endpoints**
- **premium content**
- **public search APIs**

Attackers often target:

- **login APIs**
- **payment routes**
- **recommendation APIs**
- **search endpoints**

STEP-BY-STEP IMPLEMENTATION

STEP 1 — Create Web ACL

Go to:

AWS Console



AWS WAF & Shield



Web ACLs



Create Web ACL

Configuration:

Name:

streamflix-waf

Region:

Global (for CloudFront)

Why global:

CloudFront distributions use global WAF scope.

If attached directly to ALB:

Use regional scope.

STEP 2 — Add Resource Association

Attach WAF to:

CloudFront Distribution

or

Public ALB

Best production practice:

CloudFront attachment.

STEP 3 — Configure Default Action

Default action:

Allow

Why:

WAF should only block requests matching rules.

Alternative:

Default block

Usually used only in zero-trust environments.

STEP 4 — Add AWS Managed Rules

AWS provides prebuilt managed rules.

Highly recommended in production.

Add:

AWS Managed Core Rule Set

Purpose:

Blocks common web attacks.

Includes:

- **SQL injection**
- **XSS**
- **malformed requests**

- protocol violations

Why use managed rules:

Maintained automatically by AWS security teams.

Very important for enterprise systems.

STEP 5 — Add SQL Injection Protection

Add rule:

AWSManagedRulesSQLiRuleSet

Why SQL injection matters:

Attackers try inputs like:

' OR 1=1 --

to manipulate database queries.

Example attack target:

Login APIs

Search APIs

Movie filters

Without WAF:

Malicious payload may reach backend.

With WAF:

Request blocked at edge.

STEP 6 — Add Rate Limiting Rule

Create Rate-Based Rule.

Example:

Limit:

2000 requests per 5 minutes per IP

Action:

Block

Why rate limiting matters:

Prevents:

- brute force login attacks
- API flooding
- scraper abuse

- fake traffic spikes

Real example:

Attacker sends:

50,000 login attempts

WAF automatically blocks IP.

Very important for:

POST /auth/login

STEP 7 — Add Geo Blocking

Example:

Block countries where StreamFlix is unavailable.

Rule:

IF country IN:

- blocked regions
- THEN:
- block request

Why:

Reduces attack surface.

Also useful for:

- licensing restrictions
- fraud prevention

Example:

Streaming rights available only in selected countries.

STEP 8 — Add Bot Protection

Bots can:

- scrape movie metadata
- scrape pricing
- overload APIs
- steal content indexing

Enable:

AWS Bot Control

WAF detects:

- suspicious crawlers
- automated scripts
- non-human traffic patterns

Action:

Challenge or block.

Why this matters:

Streaming platforms are heavily scraped.

STEP 9 – Protect Login APIs Separately

Create custom rule for authentication endpoints.

Example:

Path:

POST /auth/login

Rules:

- stricter rate limiting
- CAPTCHA
- IP reputation filtering

Why:

Login endpoints are highest-risk targets.

STEP 10 – Enable CAPTCHA or Challenge Actions

Instead of immediate blocking:

WAF can:

- show CAPTCHA
- require challenge validation

Useful for:

- suspicious traffic
- unknown bots
- login abuse

Why:

Avoids blocking legitimate users accidentally.

STEP 11 – Configure Rule Priority

WAF processes rules top-down.

Example:

Priority Rule

- 1 Allow trusted internal IPs**
- 2 Block malicious IPs**
- 3 SQL injection protection**
- 4 Rate limiting**
- 5 Geo blocking**
- 6 Bot filtering**

Why priority matters:

Earlier rules execute first.

STEP 12 – Enable Logging

Enable WAF logging to:

CloudWatch Logs

or

S3

or

Kinesis Firehose

Why:

Security visibility.

Logs help detect:

- **attack patterns**
- **abusive IPs**
- **scraping attempts**
- **blocked countries**
- **false positives**

Production systems ALWAYS enable WAF logs.

STEP 13 – Validate WAF

Test scenarios:

- 1. SQL Injection Test**

Example:

<https://streamflix.com/search?q=' OR 1=1 -->

Expected:

Blocked.

1. Rate Limiting Test

Send excessive requests.

Expected:

IP temporarily blocked.

1. Geo Restriction Test

Access from blocked country.

Expected:

403 Forbidden.

1. Bot Simulation

Automated scraping attempt.

Expected:

Challenge or block.

Production Considerations

Real production WAF setups usually include:

- **IP reputation lists**
- **threat intelligence feeds**
- **managed rule updates**
- **adaptive rate limiting**
- **API-specific rules**
- **separate WAF policies per environment**

Example:

Production WAF:

Strict.

Development WAF:

Less restrictive.

Why WAF Alone Is Not Enough

Important reality:

WAF is NOT a replacement for:

- **secure coding**
- **authentication**
- **IAM**
- **security groups**

- input validation

WAF is an additional protection layer.

Defense-in-depth architecture is required.

How WAF Fits Into StreamFlix Security Model

Full security chain:

Internet

↓

CloudFront

↓

AWS WAF

↓

Public ALB

↓

Frontend

↓

Internal ALB

↓

Backend Services

↓

Database

Each layer reduces risk.

Why WAF Is Essential for StreamFlix

Without WAF:

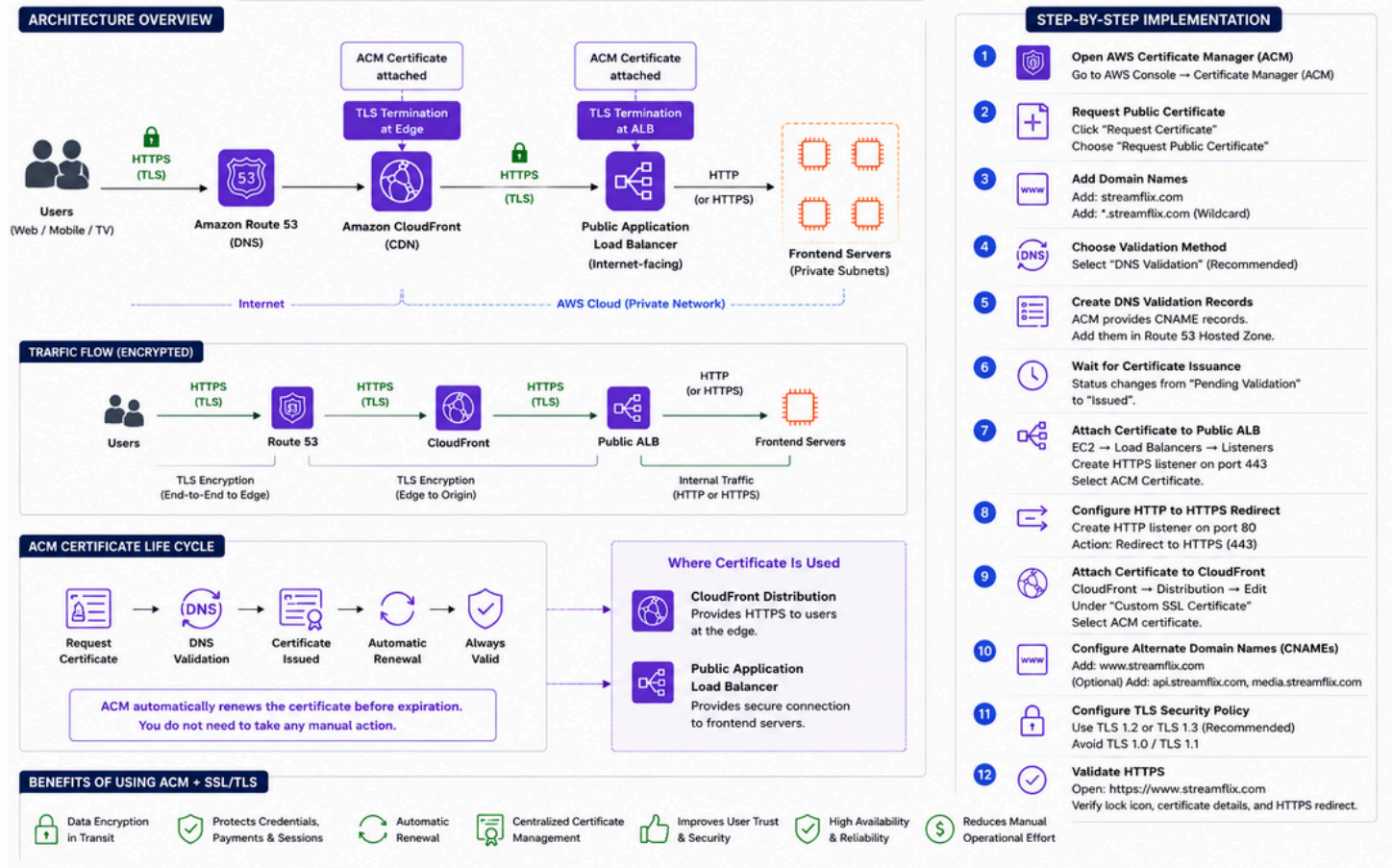
- malicious traffic reaches backend
- infrastructure costs rise
- APIs become vulnerable
- DDoS exposure increases
- bots overload services

With WAF:

- attacks filtered early
- backend protected
- traffic quality improves
- operational stability improves
- security posture strengthens

13. SSL/TLS with AWS ACM

SSL/TLS WITH AWS ACM – STREAMFLIX



Why SSL/TLS Is Required in StreamFlix

StreamFlix is a public streaming platform where users continuously exchange sensitive data with the application.

Examples include:

- login credentials
- JWT/session tokens
- subscription details
- payment requests
- profile information
- API requests
- streaming session metadata

If traffic is transmitted over plain HTTP:

- attackers can intercept traffic
- credentials can be stolen
- sessions can be hijacked
- payment requests can be manipulated

This is why HTTPS is mandatory.

HTTPS uses SSL/TLS encryption to secure communication between users and StreamFlix infrastructure.

Actual Traffic Flow with HTTPS

User Browser

↓ HTTPS

CloudFront

↓ HTTPS

Public ALB

↓ HTTP/HTTPS

Frontend Servers

Why Encryption Is Needed at Multiple Layers

CloudFront handles internet edge communication.

Public ALB handles:

- **secure origin traffic**
- **centralized TLS termination**
- **secure backend forwarding**

Even though frontend servers are private, user-facing traffic must remain encrypted until it safely enters the VPC infrastructure.

What SSL/TLS Actually Does

SSL/TLS encrypts:

- **request data**
- **headers**
- **cookies**
- **tokens**
- **API payloads**

This prevents:

- **man-in-the-middle attacks**
- **packet sniffing**
- **credential theft**
- **traffic tampering**

Without SSL/TLS:

Anyone intercepting traffic can read everything.

Why AWS ACM Is Used

Managing certificates manually in production is painful and risky.

Without ACM:

- certificates must be purchased manually
- renewals require manual work
- expired certificates can break the platform
- deployment becomes operationally complex

AWS ACM automates:

- certificate provisioning
- validation
- renewal
- AWS integration

This is why ACM is standard in AWS production environments.

Where ACM Certificates Are Used in StreamFlix

Certificates are attached to:

1. CloudFront Distribution
2. Public ALB

Why CloudFront Uses ACM

CloudFront is the first public edge service users connect to.

HTTPS at CloudFront ensures:

- secure global edge delivery
- browser trust
- encrypted CDN communication

Without HTTPS at CloudFront:

users would receive browser security warnings.

Why Public ALB Uses ACM

Public ALB also uses HTTPS because:

- CloudFront forwards traffic securely
- direct ALB access may still occur
- production architectures prefer encrypted origin traffic

Production-grade architecture:

User

↓ HTTPS

CloudFront

↓ HTTPS

Public ALB



Frontend Private Servers

STEP-BY-STEP IMPLEMENTATION

STEP 1 — Open AWS Certificate Manager

Go to:

AWS Console



Certificate Manager (ACM)

STEP 2 — Request Public Certificate

Click:

Request Certificate

Choose:

Request Public Certificate

Why public certificate:

Because StreamFlix serves internet users publicly.

STEP 3 — Add Domain Names

Add:

streamflix.com

***.streamflix.com**

Why wildcard certificate is important:

It automatically secures:

www.streamflix.com

api.streamflix.com

admin.streamflix.com

media.streamflix.com

without separate certificates.

This is important because large platforms use many subdomains.

STEP 4 — Choose Validation Method

Choose:

DNS Validation

Why DNS validation is preferred:

- automated
- reliable
- easier renewal
- production best practice

Avoid email validation in enterprise systems because:

- depends on email access
- harder to automate

STEP 5 – Create DNS Validation Records

ACM generates DNS validation entries.

Example:

Name:

_abcd.streamflix.com

Value:

xyz.acm-validations.aws

Add these records into Route53.

If Route53 hosts the domain:

AWS can create them automatically.

Why validation exists:

AWS must verify ownership of the domain.

STEP 6 – Wait for Certificate Issuance

Status changes:

Pending Validation

↓

Issued

Only after issuance can certificates be attached to AWS resources.

STEP 7 – Attach ACM Certificate to Public ALB

Go to:

EC2

↓

Load Balancers

↓

Public ALB

↓

Listeners

Create HTTPS Listener:

Protocol:

HTTPS

Port:

443

Select ACM Certificate.

Why listener 443:

443 is standard HTTPS port.

Now ALB accepts encrypted traffic.

STEP 8 – Configure HTTP to HTTPS Redirect

Create HTTP Listener:

Port:

80

Action:

Redirect to HTTPS

Why redirect is necessary:

Users may accidentally access:

<http://www.streamflix.com>

Without redirect:

traffic remains unencrypted.

Production systems enforce HTTPS everywhere.

Traffic behavior:

HTTP Request

↓

Automatic Redirect

↓

HTTPS Request

STEP 9 – Attach ACM Certificate to CloudFront

Go to:

CloudFront

↓

Distribution

↓

Edit

Under:

Custom SSL Certificate

Choose ACM certificate.

VERY IMPORTANT AWS DETAIL

CloudFront certificates **MUST** exist in:

us-east-1 region

Even if infrastructure is deployed in:

- ap-south-1
- eu-west-1
- us-west-2

This is one of the most important AWS CloudFront behaviors.

STEP 10 – Configure Alternate Domain Names (CNAMEs)

Add:

www.streamflix.com

Optional:

api.streamflix.com

media.streamflix.com

Why:

Users should access branded domains instead of CloudFront URLs.

Correct:

<https://www.streamflix.com>

Wrong:

<https://d123xyz.cloudfront.net>

STEP 11 – Configure TLS Security Policy

Recommended:

TLS 1.2 or TLS 1.3

Avoid:

TLS 1.0

TLS 1.1

Why:

Older TLS versions are insecure.

Production systems disable weak protocols.

STEP 12 – Validate HTTPS Functionality

Open:

<https://www.streamflix.com>

Verify:

- **browser lock icon visible**
- **no certificate warnings**
- **HTTPS redirect works**
- **certificate trusted**

Check certificate details in browser.

Expected:

Amazon-issued certificate.

How SSL/TLS Fits into StreamFlix Architecture

Complete secure flow:

User Browser

↓ HTTPS

CloudFront

↓ HTTPS

Public ALB

↓

Frontend Tier

↓

Internal Services

This ensures:

- **secure internet communication**
- **encrypted session handling**

- safe API usage
- secure payment processing

Why Automatic Renewal Is Critical

Certificates expire.

Without renewal:

- browsers show warnings
- APIs fail
- HTTPS breaks
- users lose trust

Real companies have experienced major outages because expired certificates were forgotten.

ACM solves this through automatic renewal.

Why HTTPS Is Extremely Important for Streaming Platforms

StreamFlix handles:

- authentication
- subscription payments
- premium content access
- user sessions

Without HTTPS:

- accounts can be stolen
- tokens intercepted
- payment data exposed

Modern browsers also penalize non-HTTPS sites.

For public production systems:

HTTPS is non-negotiable.

Production Enhancements

Large enterprises also configure:

- HSTS headers
- secure cookies
- strict cipher suites
- perfect forward secrecy
- TLS monitoring

Why:

To strengthen encryption posture further.

What Happens Without ACM

Without ACM:

- manual certificate deployment required
- operational overhead increases
- expiration risk increases
- downtime risk increases

ACM eliminates most of this complexity.

Why ACM Is the Correct AWS-Native Solution

ACM provides:

- free SSL certificates
- automatic renewal
- AWS integration
- ALB integration
- CloudFront integration
- simplified operations

14. External Public Application Load Balancer

Purpose

The external ALB is the primary internet-facing entry point.

Responsibilities:

- HTTPS termination
- Domain-based routing
- Traffic balancing
- WAF integration

The ALB is deployed across multiple Availability Zones.

15. Public ALB Listener Configuration

HTTP Listener (Port 80)

Action
Redirect all traffic to HTTPS

HTTPS Listener (Port 443)

Listener Rules

Priority	Condition	Action
1	Host = www.streamflix.com	Frontend Target Group
2	Host = admin.streamflix.com	Admin Frontend TG
3	Host = api.streamflix.com	API Gateway Service
4	Path = /static/*	Redirect to CloudFront
Default	All others	Return 404

16. Frontend Tier

The frontend layer runs inside private subnets.

Technologies:

- React
- Next.js SSR
- Node.js
- Nginx

Frontend servers:

- Do not have public IPs
- Receive traffic only from Public ALB
- Scale automatically using ASG

17. Why Frontend Servers Are Private

Although the application is public, frontend EC2 instances remain private because:

- Only ALB needs public exposure
- Backend access remains isolated
- Direct attacks on EC2 are prevented
- Security posture improves significantly

This is standard enterprise architecture.

18. Internal Application Load Balancer

Purpose

The Internal ALB manages communication between frontend services and backend microservices.

The Internal ALB is:

- Private
- Accessible only inside VPC
- Not internet-facing

19. Why Internal ALB Is Needed

Without internal ALB:

- Frontend must manage backend IPs manually
- Autoscaling becomes difficult
- Service discovery becomes complex
- Health checking becomes unreliable

The Internal ALB solves:

- Dynamic routing
- Service discovery
- Load balancing
- Internal failover
- Health checks

20. Internal ALB Listener Configuration

Internal ALB operates primarily on HTTPS (443).

21. Internal ALB Routing Rules

Auth Service Rules

Priority	Path	HTTP Method	Target Group
1	/auth/*	POST	Auth-Service-TG
2	/auth/*	DELETE	Auth-Service-TG

Examples:

POST /auth/login

POST /auth/signup

DELETE /auth/logout

Movie Service Rules

Priority	Path	HTTP Method	Target Group
3	/movies/*	GET	Movie-Read-TG
4	/movies/*	POST	Movie-Write-TG
5	/movies/*	DELETE	Admin-Movie-TG

Examples:

GET /movies

POST /movies

DELETE /movies

Read and write workloads are separated for scalability.

Payment Service Rules

Priority	Path	HTTP Method	Target Group
6	/payments/*	POST	Payment-Service-TG

Example:

POST /payments/subscribe

22. Microservices Architecture

Backend services are fully isolated microservices.

Services include:

- **Auth Service**
- **Movie Service**
- **Payment Service**
- **Recommendation Engine**
- **User Profile Service**
- **Streaming Metadata Service**

Each service:

- **Runs in private subnet**
- **Has dedicated target groups**
- **Can scale independently**

23. Target Groups

Each service is attached to a target group.

Examples:

Target Group	Purpose
Frontend-TG	Frontend servers
Auth-Service-TG	Authentication APIs
Movie-Read-TG	Read-heavy movie APIs
Movie-Write-TG	Admin write operations
Payment-Service-TG	Billing APIs

24. Health Checks

ALB continuously performs health checks.

If an instance fails:

- It is removed from traffic rotation
- Auto Scaling launches replacements

This improves fault tolerance.

25. Auto Scaling Groups (ASG)

Each application tier uses ASG.

Frontend ASG

Parameter	Value
Min	2
Desired	4
Max	10

Backend ASG

Parameter	Value
Min	4
Desired	8
Max	30

Scaling metrics:

- CPU usage
- Request count
- Memory utilization
- ALB target response time

26. Amazon Machine Images (AMI)

Custom AMIs contain:

- **Ubuntu**
- **Node.js runtime**
- **Nginx**
- **Monitoring agents**
- **Preconfigured services**

Benefits:

- **Faster scaling**
- **Consistent deployments**
- **Reduced bootstrapping time**

27. Launch Templates

Why Launch Templates Are Needed in StreamFlix

StreamFlix uses Auto Scaling Groups (ASGs) to automatically launch and terminate EC2 instances during traffic spikes and failures.

But ASG itself does not know:

- **which AMI to use**
- **which instance type to launch**
- **which security group to attach**
- **which IAM role to assign**
- **which startup scripts to run**

This is where Launch Templates become essential.

A Launch Template acts as the blueprint for creating EC2 instances automatically.

Without Launch Templates:

Every EC2 launch would require manual configuration.

That would make:

- **scaling inconsistent**
- **deployments error-prone**
- **automation difficult**
- **infrastructure unreliable**

What Launch Templates Define

Launch Templates store:

- **AMI ID**
- **instance type**
- **security groups**

- IAM role
- key pair
- storage configuration
- user-data scripts
- network settings

ASG uses this template whenever new instances must be launched.

How Launch Templates Fit into StreamFlix Architecture

Traffic spike occurs

↓

ASG detects high load

↓

ASG uses Launch Template

↓

New EC2 launched automatically

↓

Instance joins Target Group

↓

ALB starts routing traffic

Why This Is Important in Production

Imagine StreamFlix launches a new movie.

Traffic suddenly jumps from:

10,000 users

to

500,000 users

ASG must launch new instances immediately.

Without Launch Templates:

- manual setup impossible
- inconsistent server configuration
- deployment delays
- scaling failures

Launch Templates ensure every new server is identical and production-ready.

Relationship Between AMI and Launch Template

AMI provides:

Base machine image.

Launch Template provides:
Complete EC2 launch configuration.

Example:

AMI:
Ubuntu + Node.js + Nginx + StreamFlix app

Launch Template:

- **use this AMI**
- **launch t3.medium**
- **attach frontend SG**
- **attach IAM role**
- **run startup script**

Both work together.

Why Launch Templates Are Better Than Launch Configurations

Older AWS feature:
Launch Configurations

Modern production standard:
Launch Templates

Why Launch Templates are preferred:

- **versioning support**
- **multiple instance types**
- **advanced networking**
- **Spot instance integration**
- **more flexible**
- **actively supported by AWS**

Production systems should use Launch Templates.

STEP-BY-STEP IMPLEMENTATION

STEP 1 — Prepare EC2 Instance

Before creating Launch Template:
configure one production-ready EC2 instance.

Example frontend server includes:

- **Ubuntu**
- **Node.js**
- **Nginx**

- application code
- monitoring agents
- CloudWatch agent

Why:

This instance becomes the golden base image.

STEP 2 — Create AMI

Go to:

EC2

↓

Instances

↓

Select Configured Instance

↓

Actions

↓

Image and Templates

↓

Create Image

Name:

streamflix-frontend-ami

Why AMI is required:

Launch Template references the AMI.

Without AMI:

ASG cannot create identical servers.

STEP 3 — Open Launch Templates

Go to:

EC2

↓

Launch Templates

↓

Create Launch Template

STEP 4 — Configure Basic Details

Example:

Launch Template Name:
streamflix-frontend-template

Description:
Frontend Auto Scaling Template

Why naming matters:
Production environments often have:

- **frontend templates**
- **backend templates**
- **worker templates**
- **staging templates**
- **blue/green deployment templates**

Proper naming avoids confusion.

STEP 5 — Select AMI

Choose:

streamflix-frontend-ami

Why:
Every new instance launched by ASG should contain:

- **frontend application**
- **dependencies**
- **correct configurations**

STEP 6 — Select Instance Type

Example:

t3.medium

Why instance type matters:
Determines:

- **CPU**
- **RAM**
- **networking performance**
- **cost**

Frontend services usually need:
moderate CPU + memory.

Backend services may require:
larger compute.

Production teams choose based on:

- benchmarking
- traffic patterns
- memory profile
- cost optimization

STEP 7 — Configure Key Pair

Select SSH key pair.

Example:

streamflix-key

Why:

Allows administrative access.

Optional:

**Some production environments disable SSH completely and use:
AWS Systems Manager Session Manager**

STEP 8 — Configure Security Groups

Attach appropriate security group.

Example:

Frontend Security Group

Rules:

Allow traffic only from Public ALB.

Why:

New instances must automatically inherit correct security posture.

Without SG assignment:

instances may become unreachable or insecure.

STEP 9 — Configure IAM Role

Attach IAM instance profile.

Example:

streamflix-frontend-role

Possible permissions:

- CloudWatch logging
- S3 access
- Secrets Manager access

- Parameter Store access

Why IAM role matters:

Applications should not use hardcoded AWS credentials.

Production best practice:

Use IAM roles for temporary credentials.

STEP 10 — Configure Storage

Example:

Root Volume:

30 GB gp3 SSD

Why:

Streaming platforms generate:

- logs
- temporary files
- application cache

Storage must be sufficient.

Production systems also optimize:

- IOPS
- throughput
- encryption

STEP 11 — Configure User-Data Script

User-data runs automatically during EC2 startup.

Example:

```
#!/bin/bash
```

```
cd /app
```

```
npm install
```

```
npm start
```

Possible use cases:

- pull latest code
- install packages
- register monitoring
- start services
- fetch secrets

Why user-data is important:

New instances become operational automatically.

Without automation:

manual setup would delay scaling.

STEP 12 – Configure Network Settings

Subnet selection usually handled by ASG.

But Launch Template defines:

- **public IP behavior**
- **network interfaces**
- **security posture**

For StreamFlix frontend:

Public IP:

Disabled

Why:

Frontend instances remain private.

STEP 13 – Enable Detailed Monitoring

Enable:

Detailed CloudWatch Monitoring

Why:

Better scaling metrics.

ASG decisions become more accurate.

STEP 14 – Create Launch Template

Review settings.

Click:

Create Launch Template

Now reusable infrastructure blueprint exists.

STEP 15 – Create Auto Scaling Group Using Launch Template

Go to:

EC2

↓

Auto Scaling Groups

↓

Create ASG

Select:

streamflix-frontend-template

Now ASG automatically launches instances using this template.

How Launch Templates Work with ASG

ASG Scaling Event

↓

Launch Template Used

↓

New EC2 Created

↓

Target Group Registration

↓

ALB Health Check

↓

Traffic Starts Flowing

Why This Is Critical for High Availability

If instance crashes:

ASG launches replacement automatically.

Launch Template guarantees:

replacement instance is identical.

Without Launch Templates:

recovery would require manual provisioning.

Production Enhancements

Real enterprise environments also use:

Template versioning

Blue/green deployments

Mixed instance policies

Spot + On-Demand combinations

Encrypted EBS volumes

Immutable infrastructure patterns

Launch Template Versioning

Very important feature.

Example:

Version 1:

Node.js 18

Version 2:

Node.js 20

ASG can gradually migrate to newer versions.

Why:

Safe deployments and rollback capability.

Blue/Green Deployment Example

Old Launch Template:

streamflix-template-v1

New Launch Template:

streamflix-template-v2

Traffic gradually shifts to new instances.

Why:

Zero downtime deployments.

How Launch Templates Improve StreamFlix Architecture

Without Launch Templates:

- **scaling inconsistent**
- **deployments slower**
- **recovery manual**
- **higher operational risk**

With Launch Templates:

- **identical infrastructure**
- **automated scaling**
- **faster recovery**
- **easier deployments**
- **production consistency**

How Launch Templates Fit into Complete StreamFlix Architecture

User Traffic

↓

Public ALB

↓

ASG detects load

↓

Launch Template creates EC2

↓

New instance joins Target Group

↓

Traffic distributed automatically

Why Launch Templates Are Essential in Production

At scale:

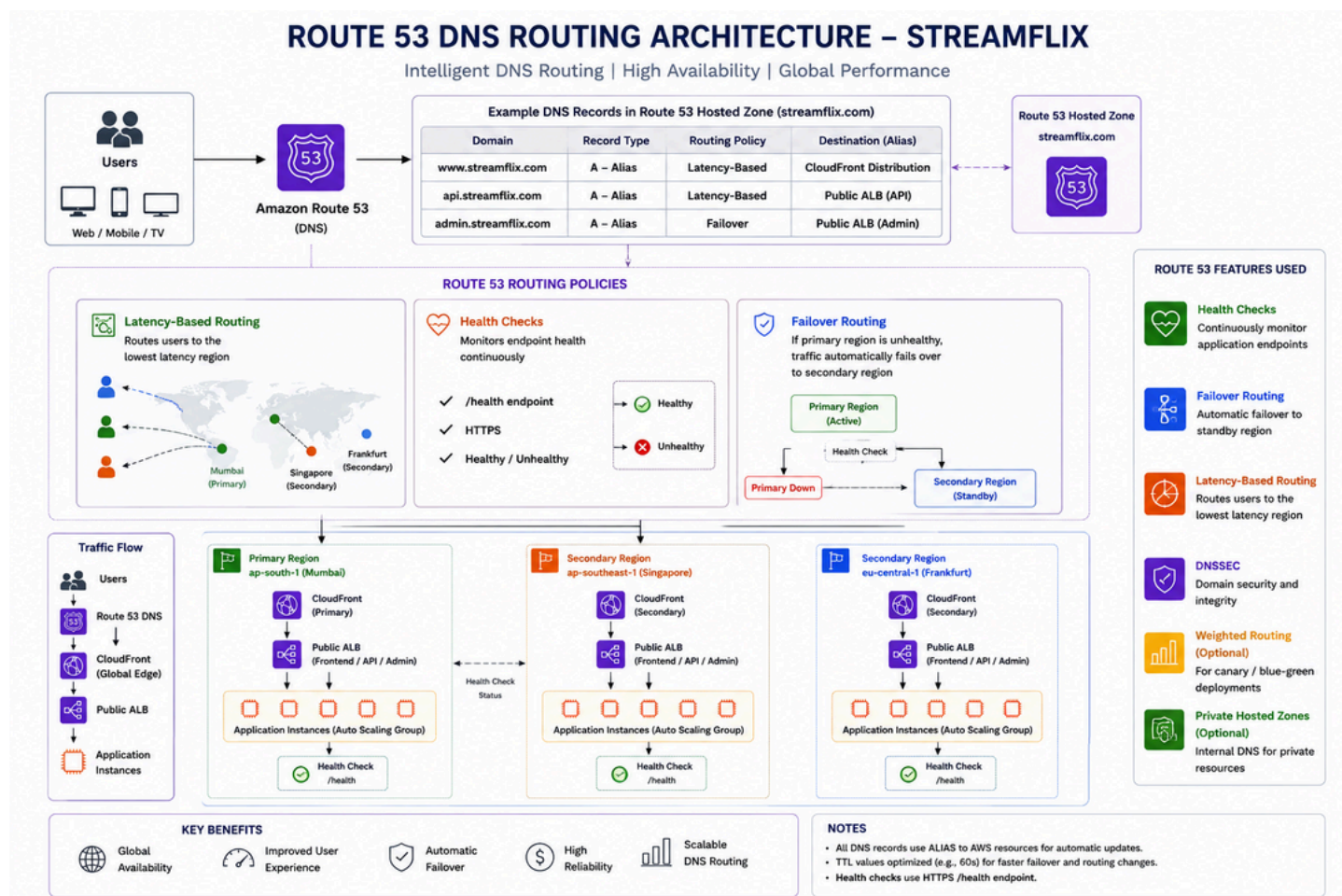
manual infrastructure creation becomes impossible.

Launch Templates provide:

- standardization
- automation
- reliability
- repeatability

They are one of the core foundations of modern cloud-native infrastructure automation.

28. Amazon Route 53



StreamFlix is a global streaming platform accessed by users through domains such as:

- www.streamflix.com
- api.streamflix.com
- admin.streamflix.com

Users should never connect using:

- raw IP addresses
- ALB DNS names
- CloudFront-generated URLs

Instead, users access human-readable domains.

This is the role of DNS.

Amazon Route 53 is the DNS service responsible for translating domain names into actual AWS infrastructure endpoints.

Without Route 53:

- users cannot access applications using domains
- failover routing becomes difficult
- global traffic management becomes harder
- health-aware routing becomes impossible

Why Route 53 Is Important Specifically for StreamFlix

StreamFlix serves:

- web users
- mobile applications
- smart TV applications
- APIs
- admin dashboards

Each service requires controlled DNS routing.

Example:

www.streamflix.com

↓

Frontend Application

api.streamflix.com

↓

API Infrastructure

admin.streamflix.com

↓

Admin Dashboard

Route 53 becomes the intelligent traffic entry layer.

How Route 53 Fits into StreamFlix Architecture

User

↓

Route53 DNS Resolution

↓

CloudFront / Public ALB

↓

Frontend Services

DNS is the FIRST infrastructure layer users interact with.

What Route 53 Actually Does

Route 53 performs:

- domain name resolution
- traffic routing
- failover routing
- health-aware DNS
- latency optimization
- global traffic management

Why DNS Matters in Production

Imagine Public ALB IP changes.

Without DNS:

Users would lose connectivity.

With Route 53:

DNS automatically points to correct infrastructure.

This abstraction is critical.

STEP-BY-STEP IMPLEMENTATION

STEP 1 — Purchase or Use Domain

Example domain:

streamflix.com

Domains can be:

- purchased in Route 53
- imported from GoDaddy
- imported from Namecheap
- imported from external registrars

Why:

DNS requires ownership of domain.

STEP 2 — Open Route 53 Console

Go to:

AWS Console

↓

Route 53

STEP 3 — Create Hosted Zone

Go to:

Hosted Zones

↓

Create Hosted Zone

Configuration:

Domain Name:

streamflix.com

Type:

Public Hosted Zone

Why Public Hosted Zone:

Because StreamFlix serves internet users publicly.

What Hosted Zone Represents

Hosted Zone is the DNS container for the domain.

Inside it:

- A records
- CNAME records
- MX records
- TXT records
- health checks

are stored.

STEP 4 – Configure Name Servers

Route 53 generates NS records.

Example:

ns-123.awsdns.com

ns-456.awsdns.net

Update these name servers in domain registrar.

Why:

This delegates DNS control to Route 53.

Without this step:

Route 53 records will not work publicly.

STEP 5 – Create Alias Record for Frontend

Create record:

Record Name:

www.streamflix.com

Record Type:

A – IPv4 Address

Alias:

Enabled

Target:

Public ALB

Why Alias record instead of IP:

ALB IP addresses change dynamically.

Alias automatically tracks ALB changes.

Very important AWS production feature.

Traffic Flow

User

↓

www.streamflix.com

↓

Route53

↓

Public ALB

STEP 6 — Create API DNS Record

Example:

api.streamflix.com

↓

Public ALB

Why separate API domain:

Production systems often isolate:

- frontend traffic
- API traffic
- admin traffic

for:

- scaling
- security
- routing control

STEP 7 — Create Admin Domain Record

Example:

admin.streamflix.com

↓

Public ALB

Why separate admin domain:

Admin systems usually require:

- stricter authentication
- WAF restrictions
- IP whitelisting
- separate monitoring

Separation improves security posture.

STEP 8 — Integrate CloudFront

Production architecture usually becomes:

User

↓

Route53

↓

CloudFront

↓

Public ALB

Instead of ALB directly.

In this case:

Route53 Alias Record points to:
CloudFront Distribution

Why:

CloudFront becomes the public edge layer.

STEP 9 – Configure Health Checks

Why health checks matter:

DNS should avoid routing traffic to failed infrastructure.

Create Health Check:

Route53

↓

Health Checks

↓

Create Health Check

Example:

Monitor:

<https://www.streamflix.com/health>

If endpoint fails:

Route53 can reroute traffic.

Health checks improve availability.

STEP 10 – Configure Failover Routing

Why failover routing is important:

Entire regions can fail.

Example:

Primary Region:

Mumbai

Secondary Region:

Singapore

If Mumbai fails:

Route53 automatically routes traffic to Singapore.

Traffic Flow

Users

↓

Route53

↓

Primary Region Healthy?

↓ Yes → Mumbai

↓ No → Singapore

This is critical for disaster recovery.

STEP 11 – Configure Latency-Based Routing

Why latency routing matters:

Users should connect to nearest healthy region.

Example:

Indian Users

↓

Mumbai Region

European Users

↓

Frankfurt Region

Latency-based routing improves:

- response time
- streaming performance
- user experience

Especially important for global streaming platforms.

STEP 12 – Configure TTL Values

TTL = Time To Live

Example:

60 seconds

**Why TTL matters:
DNS caching behavior.**

Lower TTL:

- faster failover
- quicker updates

Higher TTL:

- lower DNS queries
- better caching

Production systems balance both.

STEP 13 – Validate DNS Resolution

Run:

nslookup www.streamflix.com

or

dig www.streamflix.com

Expected:

DNS resolves correctly.

Also verify:

- HTTPS works
- CloudFront reachable
- ALB reachable

Production Routing Architecture

Users

↓

Route53

↓

CloudFront

↓

WAF

↓

Public ALB

↓

Frontend Services

Why Route 53 Is More Than Just DNS

Most beginners think DNS only maps names to IPs.

In production:

Route53 becomes:

- **global traffic manager**
- **disaster recovery controller**
- **latency optimizer**
- **health-aware routing layer**

Very important architectural role.

How Health Checks Improve StreamFlix Reliability

Without health checks:

users may still get routed to failed infrastructure.

With health checks:

Route53 avoids unhealthy endpoints.

This reduces downtime significantly.

Production Enhancements

Large enterprise setups also use:

Weighted routing

Geolocation routing

Private hosted zones

Split-horizon DNS

DNS failover automation

Cross-region disaster recovery

Weighted Routing Example

90% traffic

↓

Production Environment

10% traffic

↓

New Canary Deployment

Useful for:

- **testing**
- **gradual rollouts**
- **blue/green deployments**

Geolocation Routing Example

US users

↓

US region

India users

↓

Mumbai region

Why:

Regional optimization and compliance.

Private Hosted Zones

Used for:

internal-only DNS.

Example:

internal-auth.streamflix.local

accessible only inside VPC.

Useful for:

microservices communication.

Why Route 53 Is Essential for StreamFlix

Without Route53:

- no branded domains
- no intelligent routing
- difficult failover
- poor disaster recovery
- harder scaling

With Route53:

- intelligent DNS routing
- high availability
- low-latency access
- disaster recovery support
- production-grade traffic management

Why Route 53 Is the Correct AWS Solution

Route53 integrates naturally with:

- CloudFront
- ALB
- ACM
- health checks
- failover policies

This makes it ideal for enterprise AWS architectures like StreamFlix.

29. Security Groups

Security groups enforce strict network isolation.

Public ALB Security Group

Type	Port	Source
HTTPS	443	Internet

Frontend Security Group

Type	Port	Source
HTTPS	443	Public ALB SG

Internal ALB Security Group

Type	Port	Source
HTTPS	443	Frontend SG

Backend Security Group

Type	Port	Source
HTTPS	443	Internal ALB SG

Database Security Group

Type	Port	Source
MySQL	3306	Backend SG

30. Database Layer

StreamFlix uses:

- Amazon RDS MySQL
- Multi-AZ deployment
- Read replicas
- Redis cache

Benefits:

- High availability
- Automatic backups
- Failover support
- Read scalability

31. Monitoring and Logging

Monitoring stack includes:

- Amazon CloudWatch
- CloudTrail
- ALB access logs
- VPC Flow Logs

Metrics monitored:

- Request latency
- CPU utilization
- Error rates
- Network traffic

32. High Availability Strategy

The architecture eliminates single points of failure.

Components deployed Multi-AZ:

- ALBs
- EC2 instances

- NAT Gateways
- RDS databases

Traffic automatically shifts during failures.

33. Fault Tolerance

The platform survives failures using:

- Auto Scaling replacement
- Multi-AZ redundancy
- Health checks
- Internal service failover

34. End-to-End Request Flow

User

↓

CloudFront

↓

AWS WAF

↓

Public ALB

↓

Frontend Servers

↓

Internal ALB

↓

Backend Microservices

↓

Database Layer

35. Benefits Achieved

High Availability

Zero downtime architecture using Multi-AZ deployment.

Scalability

Independent scaling for frontend and backend services.

Security

Strict subnet isolation and internal-only service communication.

Operational Efficiency

Fully automated infrastructure scaling and recovery.

Performance

Low-latency global content delivery through CloudFront.

36. Final Outcome

After migration to AWS, StreamFlix successfully transformed into a cloud-native enterprise platform capable of handling millions of concurrent users.

Key achievements include:

- **Near-zero downtime**
- **Massive horizontal scalability**
- **Secure internal microservices architecture**
- **Automated deployment and recovery**
- **Improved streaming performance globally**
- **Reduced operational complexity and infrastructure cost**

The adoption of both external and internal ALBs enabled StreamFlix to implement enterprise-grade traffic management, secure service communication, and highly scalable microservices orchestration.